PROJECT REPORT

HANDWRITTEN DIGIT RECOGNITION USING NEURAL NETWORKS

(MNIST DATASET)

An Artificial Intelligence and Machine Learning Project

SUBMITTED BY

SANKET KISHOR SUTAR

2026

PROJECT REPORT

Certificate

This is to certify that the project report entitled “Handwritten Digit Recognition Using Neural Networks (MNIST Dataset)” has been prepared by Sanket Kishor Sutar as a record of the project work carried out using the MNIST handwritten digit dataset.

The project covers the collection and exploration of the MNIST dataset, image preprocessing, neural-network model development, model training, testing and evaluation, and a simple digit-recognition prediction interface. The report records the workflow and outputs available from the project implementation.

The report is prepared in accordance with the project topic and deliverables stated in the AI Project Guide Manual, September 2026.

Date: _____

Project Supervisor / Guide: _____

Signature: _____

Student Signature: _____

PROJECT REPORT

Acknowledgement

I would like to express my sincere gratitude to everyone and every resource that supported the completion of this project, “Handwritten Digit Recognition Using Neural Networks (MNIST Dataset)”.

I acknowledge the AI Project Guide Manual, which provided the project overview, objectives, dataset information, technology requirements, workflow and expected outcomes used to guide the work.

I also acknowledge the developers and maintainers of the programming libraries and development tools used during implementation. These tools supported dataset handling, image processing, neural-network development, model training, evaluation and the interactive prediction demonstration.

Finally, I am grateful for the opportunity to work through the complete project workflow. The implementation provided practical experience with handwritten image data, preprocessing, neural-network learning, performance evaluation and prediction.

Sanket Kishor Sutar

PROJECT REPORT

Abstract

The Handwritten Digit Recognition project is an Artificial Intelligence and Machine Learning project focused on recognizing handwritten numerical digits from 0 to 9 using a Neural Network model.

The project uses the MNIST Handwritten Digits Dataset, which contains 70,000 grayscale images of handwritten digits. Each image has dimensions of 28 × 28 pixels. The dataset consists of 60,000 training images and 10,000 testing images.

The main objective is to develop an intelligent model capable of identifying and classifying handwritten numbers. The project involves loading and exploring the dataset, preprocessing image values through normalization and reshaping where required, designing and configuring a neural-network architecture, and training the model while monitoring accuracy and loss.

The trained model is tested on unseen images and its performance is examined using accuracy, loss, precision, recall, F1-score and a confusion matrix. A simple digit-recognition system is also deployed to allow users to input handwritten digits and view the predicted output.

The final CNN run documented in the supplied results achieved 99.40% test accuracy on 10,000 test images, exceeding the manual's expected outcome of accuracy above 95%.

Keywords
MNIST, handwritten digit recognition, Artificial Intelligence, Machine Learning, Neural Network, CNN, image classification, image preprocessing, model training, evaluation.

CHAPTER 1

Introduction

1.1 Background

Artificial Intelligence and Machine Learning provide methods for developing systems that identify patterns in data and produce predictions or classifications. Image classification is one such task: an image is supplied to a model, and the model assigns it to one of a set of defined classes.

Handwritten digit recognition is a practical example of image classification. The input is an image containing a handwritten numerical digit, and the expected output is one of ten labels, from 0 through 9. The task requires a model to learn visual patterns despite differences in handwriting style, shape, thickness and position.

1.2 Project Overview

The MNIST Digit Classification project is described in the project manual as a beginner-friendly Artificial Intelligence and Machine Learning project. It focuses on recognizing handwritten digits using a Neural Network model trained on the MNIST dataset.

The work follows the project workflow stated in the manual: data collection, data preprocessing, model development, model training, testing and evaluation, and prediction. The resulting system accepts handwritten digit input and displays a predicted output.

CHAPTER 1

Introduction (continued)

1.3 Problem Statement

The project addresses the task of automatically identifying a handwritten numerical digit from an image. A digit image must be interpreted and assigned to the correct class among ten possible labels (0–9). The system must learn from labelled examples and then classify unseen test images.

1.4 Project Objectives

- Understand the fundamentals of Neural Networks.
- Learn image classification techniques.
- Train a model using the MNIST dataset.
- Evaluate model accuracy and performance.
- Deploy a simple digit recognition system.

1.5 Project Scope

The project scope is limited to the handwritten digit-recognition task described in the manual. The dataset contains grayscale digit images with 28 × 28 pixel dimensions and ten classes. The implementation covers image loading and exploration, normalization, neural-network development, training, testing, evaluation and a simple prediction interface.

CHAPTER 1

Dataset Information

The project uses the MNIST Handwritten Digits Dataset. The manual specifies 70,000 images in total, divided into 60,000 training samples and 10,000 testing samples. Each image is a 28 × 28 pixel grayscale image, and labels represent digits from 0 to 9.

Dataset property	Description
Dataset name	MNIST Handwritten Digits Dataset
Total images	70,000
Training samples	60,000
Testing samples	10,000
Image dimensions	28 × 28 pixels
Image format	Grayscale
Digit classes	0, 1, 2, 3, 4, 5, 6, 7, 8, 9

The training set is used to develop the model. The separate testing set is reserved for measuring performance on unseen images. The dataset labels allow the model predictions to be compared with the correct digit values.

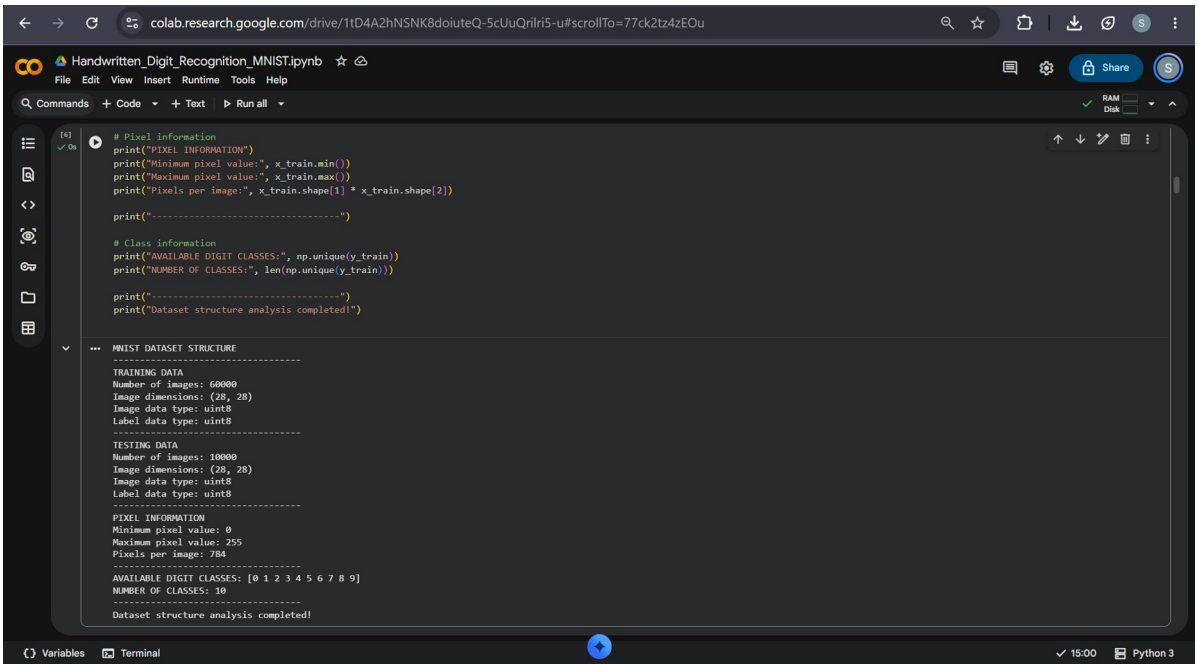


Figure 1.1. MNIST dataset structure, training and testing counts, image dimensions, pixel range and digit labels.

CHAPTER 1

Technologies Required

The project manual specifies Python as the programming language and lists the libraries and development environments below. These technologies support data handling, visualisation, neural-network model development and project execution.

Category	Technologies specified in the manual
Programming language	Python
Libraries	NumPy, Pandas, Matplotlib, TensorFlow, Keras, Scikit-learn
Development environments	Jupyter Notebook, Google Colab, Visual Studio Code (VS Code)

1.6 Role of the technologies in the project

- Python provides the programming environment for the project implementation.
- NumPy and Pandas support numerical and tabular data handling.
- Matplotlib supports the display of image samples and evaluation visualisations.
- TensorFlow and Keras provide the neural-network model development and training framework.
- Scikit-learn provides classification metrics and evaluation utilities.

Google Colab is used as the notebook-based development environment for the demonstrated workflow. The code and outputs are saved in a Jupyter notebook format.

CHAPTER 2

Literature Review

2.1 Neural Networks

The project manual identifies Neural Networks as the central model family for the handwritten digit-recognition task. A neural network is designed to learn patterns from examples by adjusting its parameters during training. In a classification setting, the learned model maps an input image to one of the defined output classes.

The manual requires the project to design a neural-network architecture and configure its input, hidden and output layers. The final implementation uses a Convolutional Neural Network (CNN), which applies convolutional operations to learn spatial patterns in image data.

2.2 Image Classification

Image classification involves assigning an image to a class based on its visual content. In the MNIST task, each input image belongs to one of ten digit classes. Classification quality is assessed by comparing model predictions with the known labels of the test images.

The manual identifies image classification as a learning objective and describes the need to test the model on unseen images and generate performance metrics.

CHAPTER 2

Literature Review (continued)

2.3 MNIST as a Handwritten Digit Dataset

MNIST provides a labelled collection of handwritten digit images suitable for demonstrating image classification. The project manual specifies the dataset size, image dimensions, grayscale format and ten digit classes. The separation into training and testing sets supports model development and independent evaluation.

2.4 Image Preprocessing

The manual includes normalization of image values and reshaping of image dimensions where required as the two main preprocessing activities. Normalization changes the scale of pixel values to a consistent numerical range. Reshaping arranges image data into the input dimensions expected by the selected model architecture.

2.5 Model Training, Evaluation and Prediction

The project workflow requires the model to be trained using training data while monitoring accuracy and loss. It then requires testing on unseen images and generating performance metrics. The final prediction phase allows a user to supply handwritten digits and view the model output.

CHAPTER 3

Methodology

3.1 Overall Methodology

The project follows the six-phase workflow stated in the MNIST section of the project manual. Each phase produces the data, model or output needed by the next phase. The overall process begins with collecting and exploring the dataset and concludes with a simple digit-recognition prediction system.

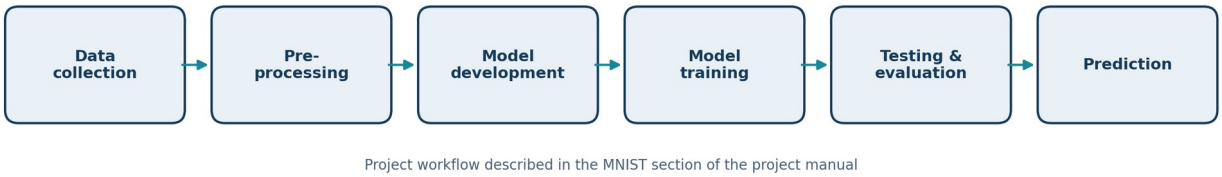


Figure 3.1. Six-phase project workflow based on the project manual.

Phase	Main activities specified in the manual
1. Data collection	Load the MNIST dataset; explore image samples.
2. Data preprocessing	Normalize image values; reshape dimensions if required.
3. Model development	Design a neural-network architecture; configure input, hidden and output layers.
4. Model training	Train using training data; monitor accuracy and loss.
5. Testing and evaluation	Test on unseen images; generate performance metrics.
6. Prediction	Allow users to input handwritten digits; display predicted output.

CHAPTER 3

Methodology (continued)

3.2 Phase 1: Data Collection

The MNIST dataset is loaded into the notebook using the dataset loader available in the TensorFlow/Keras environment. The image arrays and label arrays are separated into training and testing sets. The project checks array dimensions, number of samples, image dimensions, pixel range and available digit labels.

3.3 Phase 2: Data Preprocessing

The original MNIST pixels are represented as grayscale intensity values in the range 0 to 255. The project converts the pixel arrays to floating-point values and divides by 255 so that normalized values lie between 0 and 1. For the CNN, an additional channel dimension is included so that the input shape is 28 × 28 × 1.

The preprocessing verification checks the resulting dimensions, data type and minimum/maximum values. These checks help confirm that training, validation and testing images use a consistent format.



Figure 3.2. Pixel normalization from the original 0–255 range to normalized floating-point values.

CHAPTER 3

Methodology (continued)

3.4 Phase 3: Model Development

The model is configured to accept image data and produce ten output scores, one for each digit class. The final implementation uses a CNN architecture for image classification.

3.5 Phase 4: Model Training

The training data is separated into training and validation subsets. The model is trained on the training subset while validation accuracy and loss are monitored. The training history is recorded for analysis.

3.6 Phase 5: Testing and Evaluation

After training, the model is evaluated on the separate MNIST test set. Accuracy, loss, precision, recall, F1-score and the confusion matrix are used to describe performance and identify misclassified examples.

3.7 Phase 6: Prediction

The prediction system accepts a handwritten digit image or a drawing. The input is preprocessed into the same image format used by the model. The model returns scores for the ten digit classes, and the class with the highest score is displayed as the predicted digit.

CHAPTER 4

System Design

4.1 System Overview

The system is organised as a sequence of connected stages. The dataset is loaded and explored, image values are preprocessed, the neural network is developed and trained, and the model is evaluated on test data. The prediction interface uses the trained model to classify a user-provided digit image.

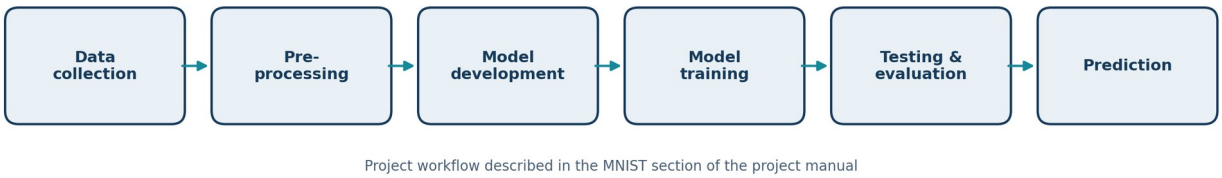


Figure 4.1. High-level system flow corresponding to the six project phases.

4.2 System Input and Output

Element	Description
Input data	28 × 28 grayscale image of a handwritten digit.
Preprocessing	Pixel normalization and dimension formatting for the model.
Model	Trained neural-network classifier with ten output classes.
Output	Predicted digit from 0 to 9 with class scores / confidence display.

CHAPTER 4

System Design (continued)

4.3 Final CNN Architecture

The final enhanced classifier uses a CNN with an input shape of 28 × 28 × 1 and ten output classes. The architecture applies convolutional layers to learn image features, batch normalization to support stable training, max-pooling to reduce spatial dimensions, dropout to reduce over-reliance on individual activations, and dense layers to produce the final class scores.

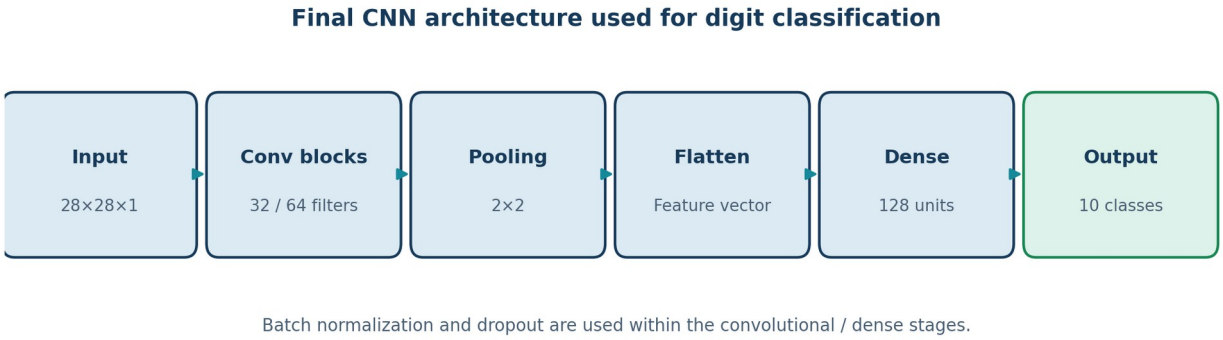


Figure 4.2. Simplified representation of the final CNN classification path.

Stage	Configuration
Input	28 × 28 × 1 grayscale image
Convolutional feature extraction	32-filter and 64-filter convolution stages with ReLU
Normalization / regularization	Batch normalization and dropout
Pooling	2 × 2 max-pooling stages
Dense representation	128-unit dense layer
Output	10-unit softmax layer for digits 0–9

The final model has 469,098 parameters according to the model summary captured during implementation.

CHAPTER 5

Implementation

5.1 Dataset Loading

The first implementation stage loads the MNIST images and labels and separates the data into training and testing sets. Dataset verification confirms 60,000 training images and 10,000 testing images. The images have dimensions of 28 × 28 pixels and the labels cover all ten digits.

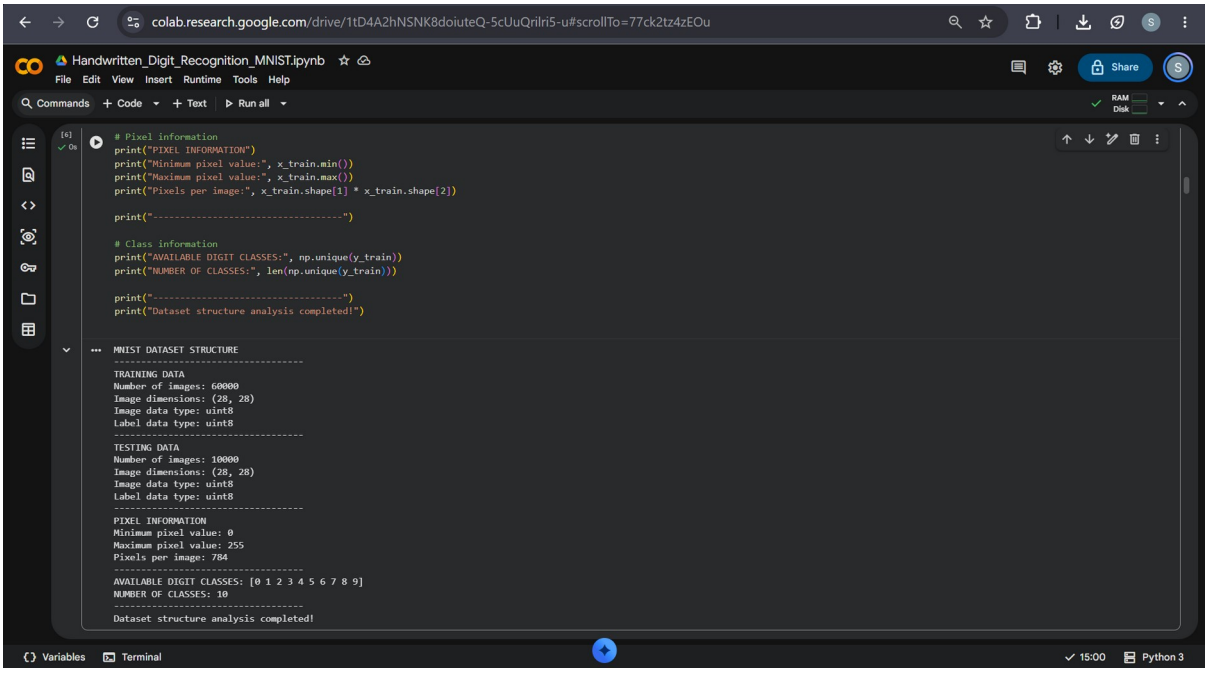


Figure 5.1. Dataset structure inspection showing the 60,000 training and 10,000 testing images.

The successful loading check establishes the dataset arrays used in the subsequent exploration and preprocessing stages.

CHAPTER 5

Implementation (continued)

5.2 Dataset Structure and Distribution

The dataset is explored to inspect training and testing dimensions, data types, pixel range and class labels. The digit distribution analysis reports the number and percentage of images for each digit class in the training and test sets.

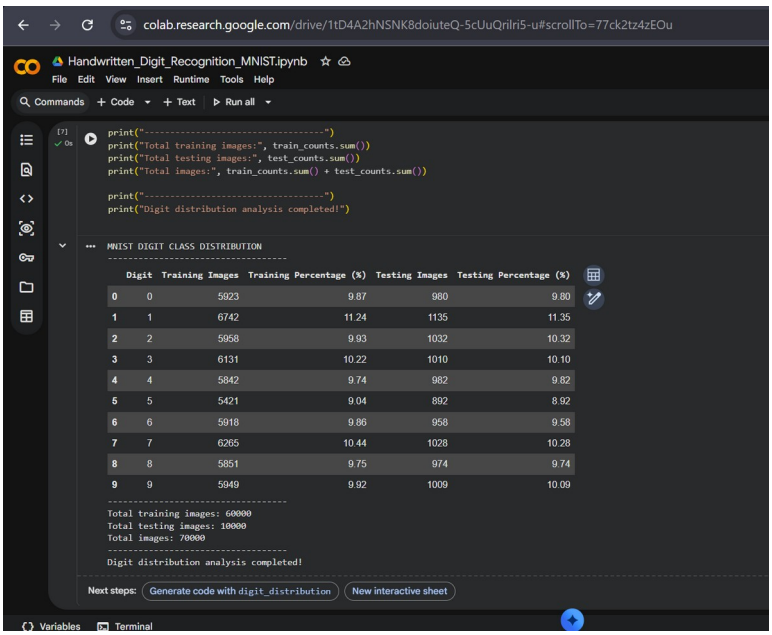


Figure 5.2. Distribution of the ten digit classes across the training and testing sets.

CHAPTER 5

Implementation (continued)

5.3 Image Exploration and Preprocessing

Sample images are displayed to inspect the handwritten digit classes. An individual image is also visualised to examine its label, shape and pixel information. Image normalization converts pixel values from the original integer range of 0 to 255 into floating-point values between 0 and 1.

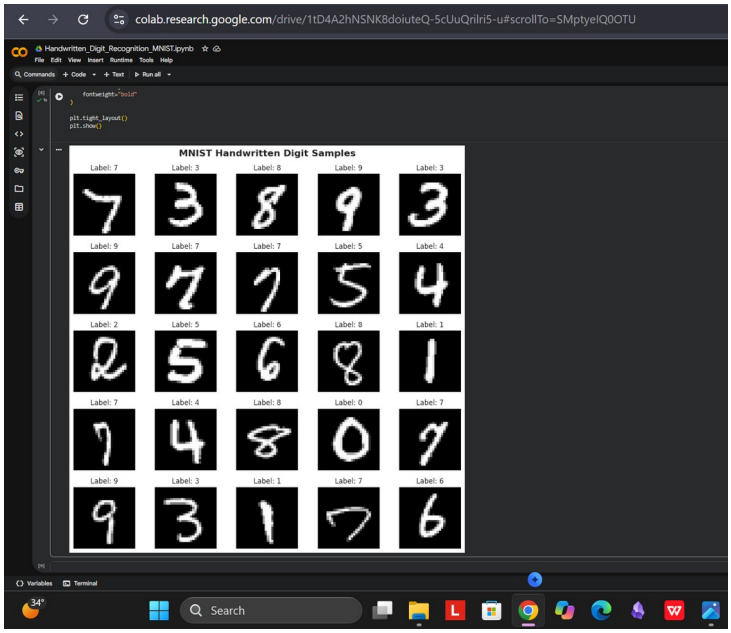


Figure 5.3. A 5 × 5 selection of handwritten MNIST digit samples.

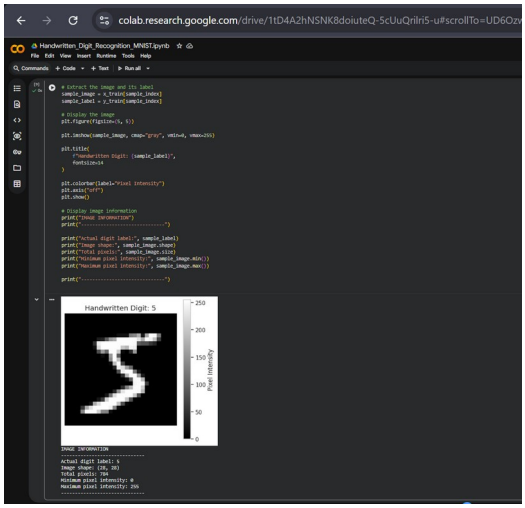


Figure 5.4. Individual handwritten digit sample with its label and pixel information.

CHAPTER 5

Implementation (continued)

5.4 Image Normalization and Preprocessing Verification

The MNIST pixel arrays are converted from unsigned integer values in the range 0 to 255 into floating-point values between 0 and 1. The CNN input preparation then adds a single channel dimension to each 28 × 28 grayscale image. Verification checks confirm the expected image shapes, data types and normalized pixel range for the training and testing data.



Figure 5.5. Original and normalized pixel-value ranges.

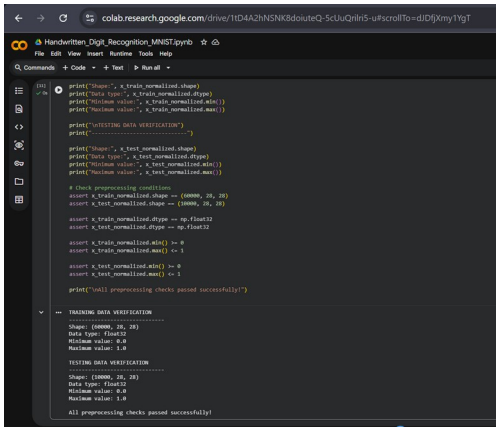


Figure 5.6. Training and testing preprocessing verification, including shapes and normalized pixel ranges.

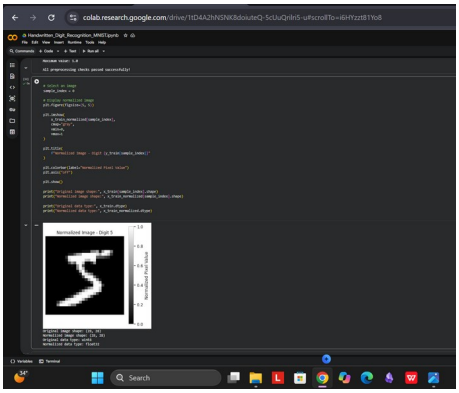


Figure 5.7. Example of a normalized MNIST image with pixel values between 0 and 1.

CHAPTER 5

Implementation (continued)

5.5 CNN Model Development and Configuration

The final model accepts a 28 × 28 × 1 grayscale image and produces ten output scores corresponding to digits 0 through 9. Its convolutional stages use 32 and 64 filters with ReLU activation. Batch normalization and dropout are used within the feature-extraction and dense stages. Max-pooling reduces spatial dimensions, and a 128-unit dense layer feeds a ten-unit softmax output layer.

```
            layers.MaxPooling2D(pool_size=(2, 2)),
            layers.Dropout(0.25),

            # FEATURE EXTRACTION
            layers.Flatten(),

            # FULLY CONNECTED LAYER
            layers.Dense(
                128,
                activation="relu"
            ),

            layers.BatchNormalization(),
            layers.Dropout(0.40),

            # OUTPUT LAYER: DIGITS 0 TO 9
            layers.Dense(
                10,
                activation="softmax"
            )

        ],
        name="MNIST_Improved_CNN"
    )

    print("Improved CNN architecture created successfully!")
    print("Input image shape: 28 x 28 x 1")
    print("Output classes: 10")
```

```
Improved CNN architecture created successfully!
Input image shape: 28 x 28 x 1
Output classes: 10
```

Figure 5.8. Implementation of the improved CNN architecture in the notebook.

```
    )

    print("CNN MODEL COMPILED SUCCESSFULLY!")
    print("-----------------------------------")
    print("Optimizer: Adam")
    print("Learning Rate: 0.001")
    print("Loss Function: Sparse Categorical Crossentropy")
    print("Evaluation Metric: Accuracy")
    print("-----------------------------------")
```

```
CNN MODEL COMPILED SUCCESSFULLY!
-----------------------------------
Optimizer: Adam
Learning Rate: 0.001
Loss Function: Sparse Categorical Crossentropy
Evaluation Metric: Accuracy
-----------------------------------
```

Figure 5.9. CNN compilation configuration using the Adam optimizer and sparse categorical cross-entropy.

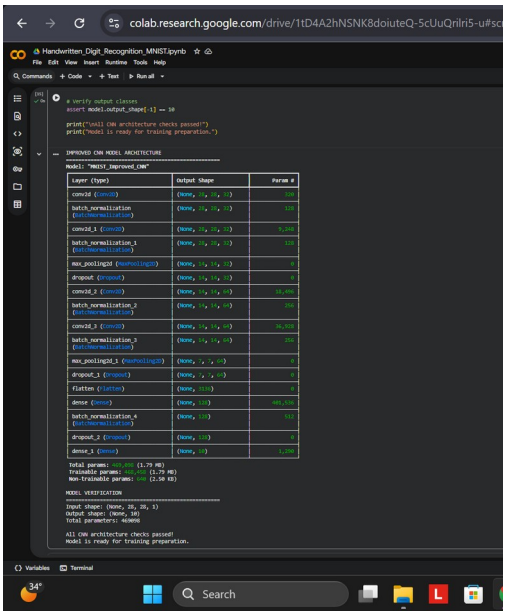


Figure 5.10. Final CNN model summary showing input/output shapes and 469,098 total parameters.

CHAPTER 5

Implementation (continued)

5.6 CNN Data Preparation and Training Process

The original 60,000-image training set is divided into 54,000 training images and 6,000 validation images. The separate test set contains 10,000 images. For the CNN, all image arrays are reshaped to 28 × 28 × 1 and checked to ensure that their values remain within the normalized range. The model is trained using a lightweight eight-epoch configuration with a batch size of 256, early stopping and learning-rate reduction callbacks.

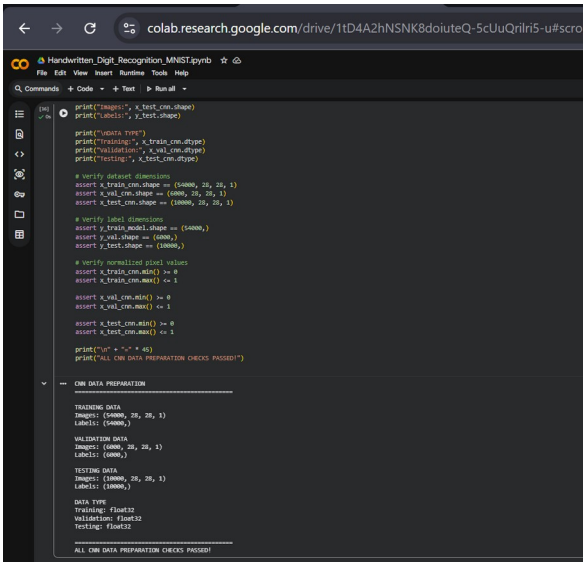


Figure 5.11. CNN training, validation and testing data shapes after preparation.

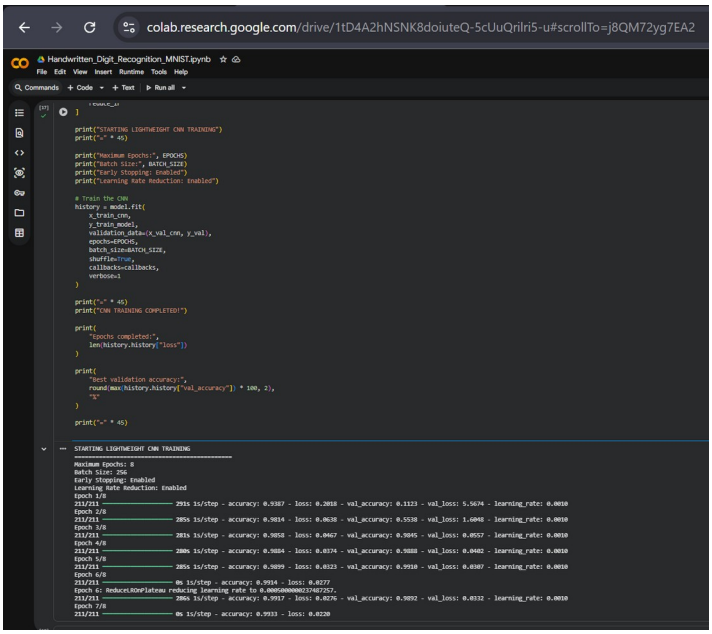


Figure 5.12. CNN training execution log showing the epoch-by-epoch training and validation progress.

CHAPTER 5

Implementation (continued)

5.7 Gradio Prediction Interface

A simple Gradio interface is used to demonstrate digit prediction. The user can draw a digit or upload an image and select a white or black brush colour according to the canvas background. The preprocessing function converts the input into the format expected by the trained CNN. The interface displays the top predicted classes, the predicted digit, a confidence value and a processed 28 × 28 image preview.

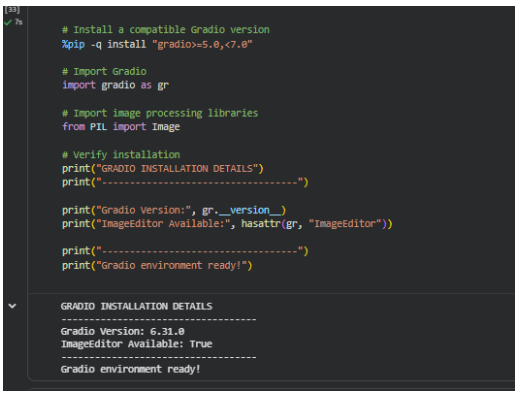


Figure 5.13. Gradio environment verification, including the installed version and ImageEditor availability.

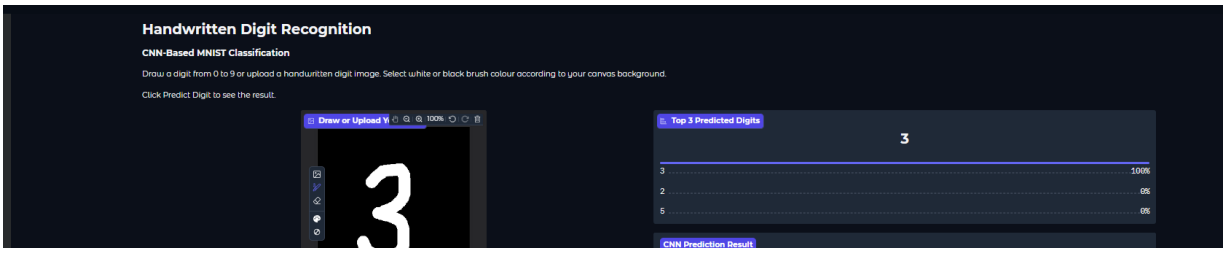


Figure 5.14. Gradio CNN prediction interface with a handwritten digit input and top-three predicted classes.



Figure 5.15. Gradio prediction result showing digit 3, confidence and the processed 28 × 28 image preview.

CHAPTER 6

Results

6.1 Training Results

The CNN training history demonstrates how training and validation accuracy changed across the eight recorded epochs. Training accuracy increased to 99.51%. Validation accuracy reached a best value of 99.17%. The best validation loss was 0.0286.

CNN training measure	Recorded value
Epochs completed	8
Best training accuracy	99.51%
Best validation accuracy	99.17%
Best validation loss	0.0286

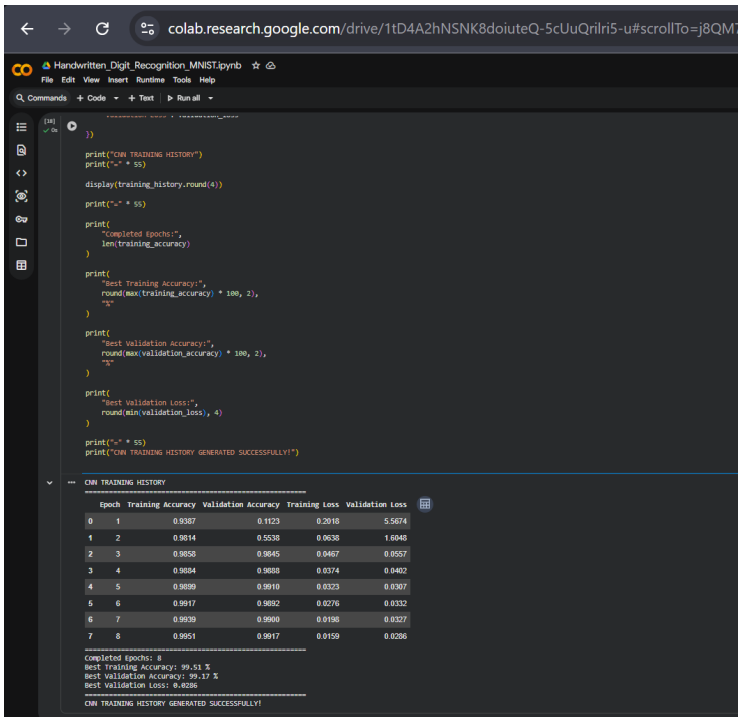


Figure 6.1. Latest CNN training history across eight epochs, including training and validation accuracy and loss.

CHAPTER 6

Results (continued)

6.2 Final CNN Test Evaluation

The final CNN was evaluated on the separate MNIST test dataset containing 10,000 images. The supplied evaluation output reports a test accuracy of 99.40% and a test loss of 0.0175. This exceeds the project manual's expected outcome of digit-recognition accuracy above 95%.

Evaluation measure	Final CNN result
Test images	10,000
Test accuracy	99.40%
Test loss	0.0175
Correct predictions	9,940
Incorrect predictions	60



Figure 6.2. Latest CNN test-set evaluation output: 99.40% accuracy and 0.0175 loss.

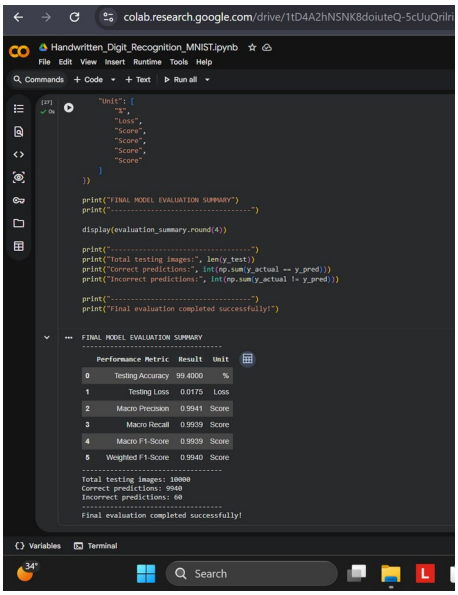


Figure 6.3. Final CNN evaluation summary with 9,940 correct predictions and 60 incorrect predictions.

CHAPTER 6

Results (continued)

6.3 Classification Report

The classification report provides class-wise precision, recall, F1-score and support. The CNN report shown in the supplied output gives an overall accuracy of 99.40%, with macro-average F1-score of 99.39% and weighted-average F1-score of 99.40%. These metrics provide a class-level view of the model's performance beyond accuracy alone.

Metric	CNN result
Accuracy	99.40%
Macro precision	99.41%
Macro recall	99.39%
Macro F1-score	99.39%
Weighted F1-score	99.40%

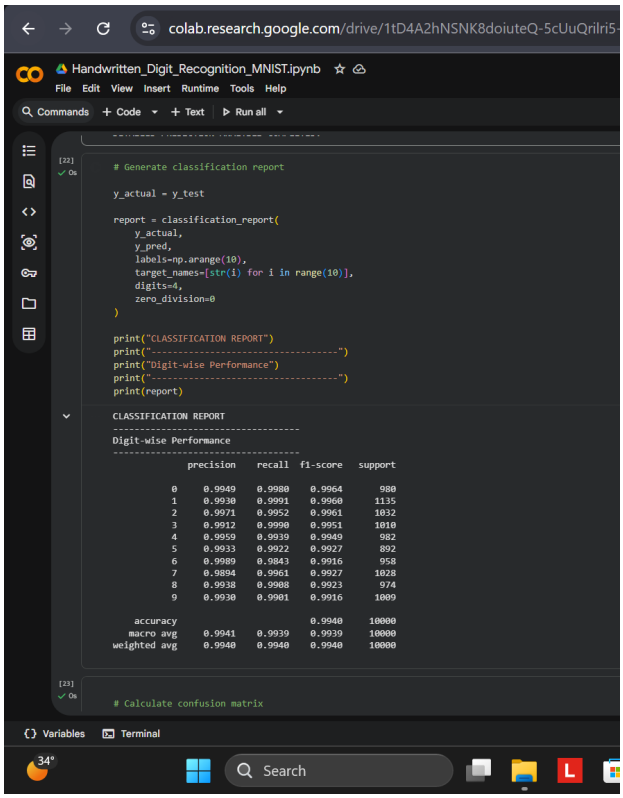


Figure 6.4. Latest CNN classification report with per-class precision, recall, F1-score and support.

CHAPTER 6

Results (continued)

6.4 Confusion Matrix

The confusion matrix compares actual digit labels with predicted digit labels for the ten classes. Values along the main diagonal represent correctly classified images. Off-diagonal values represent images assigned to a different digit class.

The final CNN confusion matrix contains 9,940 correct predictions across the 10,000 test images. The remaining 60 samples are misclassified. The strong diagonal pattern indicates that most test examples are assigned to their correct digit class.

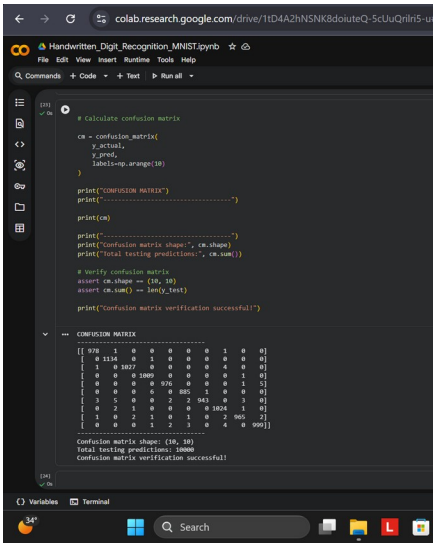


Figure 6.5. Numerical 10 × 10 confusion matrix calculated on the 10,000-image test set.

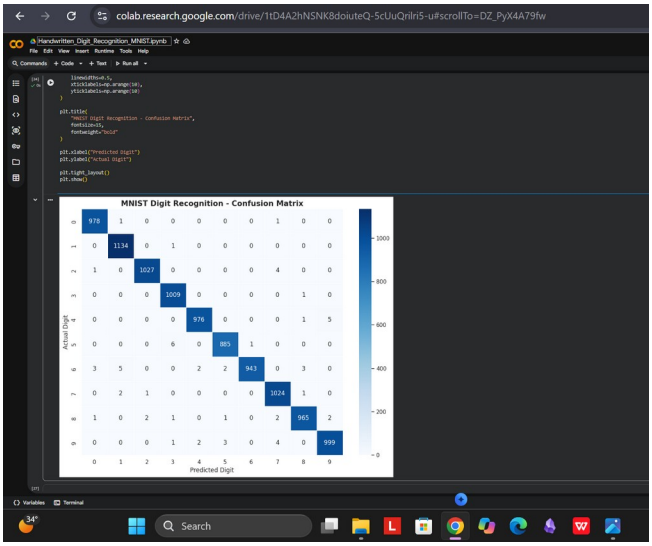


Figure 6.6. Latest CNN confusion-matrix heatmap for the ten digit classes.

CHAPTER 6

Results (continued)

6.5 Prediction Examples and Misclassification Analysis

The prediction visualisations compare the actual labels with the model outputs and display confidence values. The selected CNN prediction grid shows examples from all ten digit classes, while the individual prediction example demonstrates a correct prediction for digit 7. The error-analysis output identifies 60 misclassified images from the 10,000-image test set and displays examples of those errors.

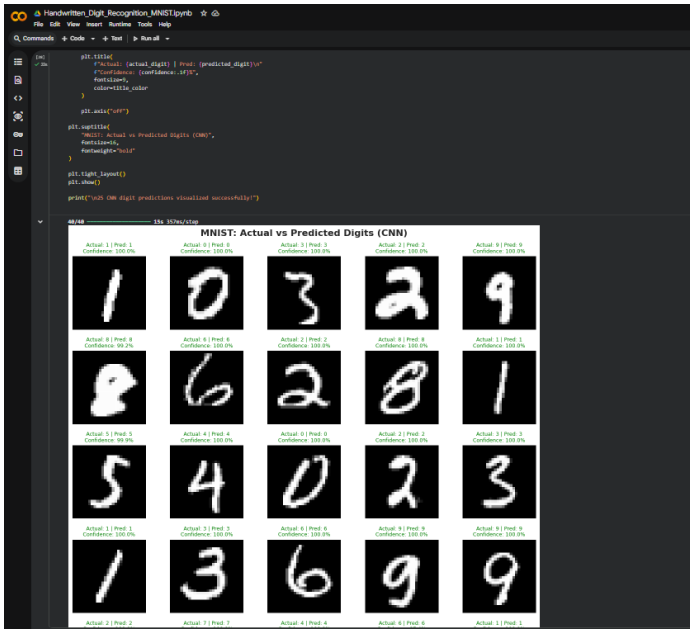


Figure 6.7. CNN prediction grid comparing actual and predicted labels for 25 selected test images.

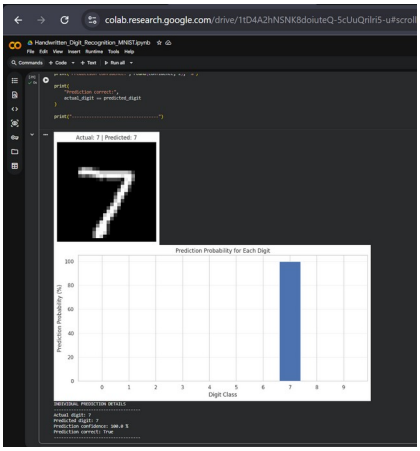


Figure 6.8. Individual CNN prediction example for digit 7 with a 100.0% displayed confidence.

CHAPTER 6

Results (continued)

6.6 Misclassification Analysis

The CNN correctly classified 9,940 of the 10,000 test images. The remaining 60 images were incorrectly classified. The examples below illustrate cases where visually similar or atypical handwriting led the model to predict a different digit. This analysis helps identify where classification errors occurred, even though the overall test accuracy was 99.40%.

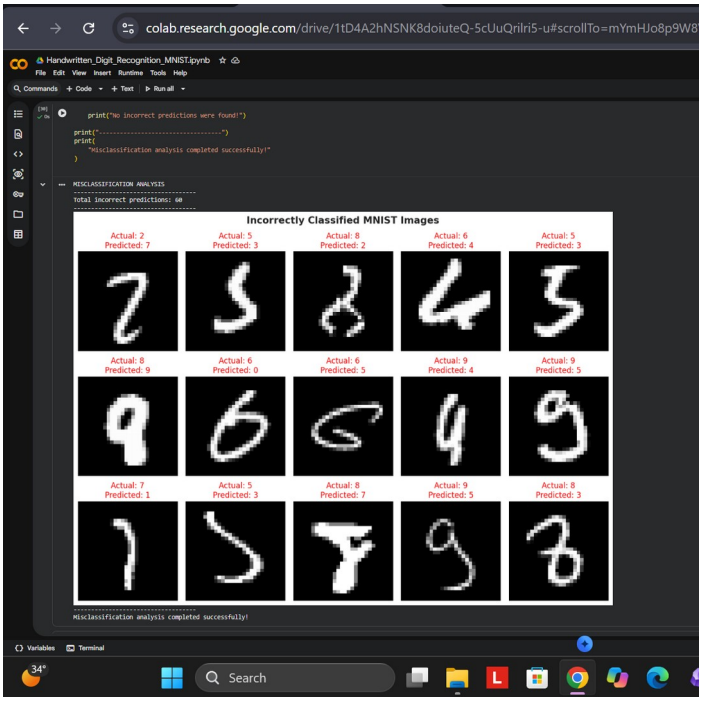


Figure 6.9. Examples of incorrectly classified CNN test images from the 60 errors in the test set.

CHAPTER 7

Conclusion

The project demonstrates the development of a handwritten digit-recognition system using the MNIST dataset and a Neural Network model. The work follows the project manual's six-phase workflow, covering dataset collection, preprocessing, model development, model training, testing and evaluation, and prediction.

The MNIST dataset was loaded and explored. Image values were normalized and the dimensions were formatted for the CNN. The final model uses convolutional layers and supporting normalization, pooling and dropout stages to classify the ten digit classes.

The final CNN run documented in the supplied results achieved 99.40% test accuracy on 10,000 unseen test images, with a test loss of 0.0175. The result exceeds the manual's expected accuracy benchmark of above 95%. Precision, recall, F1-score and the confusion matrix were also used to examine model performance.

A simple prediction interface was implemented to accept handwritten digit drawings or uploaded images and display predicted output. The project therefore covers both model development and the manual's requirement to deploy a simple digit-recognition system.

Overall outcome
The project achieved its core objective: developing and evaluating a neural-network-based system for classifying handwritten digits from 0 to 9.

CHAPTER 8

Future Scope

The project manual identifies several application areas for handwritten digit recognition. The current implementation demonstrates the core digit-classification workflow; future work may explore how the same recognition approach can be adapted for broader document and form-processing tasks.

8.1 Applications identified in the project manual

- Bank cheque processing.
- Postal code recognition.
- Optical Character Recognition (OCR) systems.
- Educational tools.
- Automated form processing.

8.2 Possible extensions

Possible future work includes testing the recognition approach on additional handwritten samples, examining how the model behaves with different writing styles and image quality, and improving the input-handling process for varied user uploads. The prediction interface may also be refined to support a broader range of digit-image sources.

These items are presented as potential extensions. They are not claimed as features already completed in the current project.

PROJECT REPORT

References

[1] AI Project Guide Manual, September 2026. “MNIST Digit Classification Using Neural Networks.” User-provided project manual. This is the primary source for the project title, overview, objectives, dataset information, technology requirements, workflow, expected outcomes, applications and deliverables.

[2] MNIST Handwritten Digits Dataset, accessed in the project implementation through the TensorFlow/Keras MNIST dataset loader. Dataset details used in this report are limited to the information and outputs documented for the project: 70,000 grayscale 28 × 28 images, including 60,000 training and 10,000 testing samples.

[3] Project implementation notebook: Handwritten_Digit_Recognition_MNIST.ipynb. User-provided notebook screenshots and recorded outputs used to describe the implementation and results.

[4] Project source code: mnist_digit_recognition.py. User-provided exported Python source file.

[5] Project evaluation and prediction screenshots supplied by the project author. These include dataset loading and analysis, image preprocessing, model development, training history, test evaluation, classification reports, confusion matrices, prediction examples and the Gradio interface.

Note on reported results
The metrics in this report are taken from the latest supplied CNN training and evaluation screenshots: 99.40% test accuracy, 0.0175 test loss, 9,940 correct predictions and 60 incorrect predictions across 10,000 test images.